Kapnet State Structure and Runtime Examples
Typed Nested Roots, Provider Sync, Member-Addressed Submission Handling, and DAO Feed Browsing

1 Abstract

This document specifies a practical state-tracking design for Kapnet based on a canonical top-level state root and a fixed set of header-visible typed subroots. It also provides runtime examples showing how a sleeping node wakes and fetches state from providers, how it handles member-addressed submissions, and how it scrolls DAO feeds under a declared browse algorithm mode.

The design goal is not merely to store state, but to make state legible, selectively fetchable, proof-friendly, operationally visible, and suitable for a provider market. The core principle is that Kapnet should not expose one giant opaque root when the system naturally decomposes into multiple state lanes with distinct persistence, query, and verification behavior.

2 Design Goal

Kapnet should track state as a *root of typed roots*. The header of a canonical state summary should expose the most important subroots directly so that relays, validators, archivists, indexers, light clients, DAOs, and agent runtimes can reason about change boundaries without descending into all state immediately.

This design supports:

- selective sync,
- lane-specific proofs,
- lane-specific subscriptions,
- better debugging and observability,
- clearer provider market boundaries,
- easier state sharding and snapshotting,
- explicit separation between history, replay, policy, capability, archive, and receipts.

3 Canonical Top-Level State

Kapnet state is defined by one canonical state shell.

3.1 State Shell

```
KapState = {
  state_version,
  scope_id,
  epoch,
  state_root,

  namespace_root,
  object_log_root,
  replay_root,
  dependency_root,
  policy_root,
  receipt_root,
  archive_root,
  capability_root
}
```

The top-level state root is derived from the ordered tuple of typed subroots.

3.2 Top-Level State Root Formula

$$\text{state_root} = H(\text{state_version} \parallel \text{scope_id} \parallel \text{epoch} \parallel \text{namespace_root} \parallel \text{object_log_root} \parallel \text{replay_root} \parallel \text{dependency}$$

The function $H(\cdot)$ should be a canonical hash function selected by the kernel specification. This document leaves the exact hash primitive abstract.

4 Why Fixed Named Subroots

The first layer of state should not be dynamically named or arbitrarily extensible. If the first layer is unstable, interoperable light verification becomes harder, provider services become less predictable, and client logic becomes more brittle.

The first layer should therefore be:

- fixed,
- few,
- ordered,
- typed,
- consensus-bearing.

Dynamic extensibility belongs one level down, inside each typed subtree.

5 Top-Level Root Semantics

5.1 Namespace Root

The namespace root commits to where state lives.

It should cover:

- room registry,
- guild registry,
- DAO registry,
- scope metadata references,
- active rule-package references per scope.

A suitable decomposition is:

```
namespace_root = H(
  room_registry_root
  || guild_registry_root
  || dao_registry_root
  || scope_rule_ref_root
)
```

5.2 Object Log Root

The object log root commits to the accepted object history for the current scope and epoch horizon.

It should track:

- accepted envelope IDs,
- canonical object ordering,
- append-only historical chunks,
- acceptance receipts or acceptance indexes.

This lane is best modeled as an append-only Merkleized log or chunked accumulator.

5.3 Replay Root

The replay root commits to consumed replay keys and uniqueness slots. It should be treated as first-class because replay state is not merely historical decoration; it directly affects correctness.

It should cover:

- consumed replay keys,
- consumed nonces,
- explicit replay markers,
- domain-specific uniqueness locks.

A sparse keyed commitment is a natural fit here.

5.4 Dependency Root

The dependency root commits to declared dependency structure.

It should cover:

- object-to-dependency summary,
- accepted dependency edges,
- dependency closure summaries where materialized,
- optional dependency chunk indexes.

This lane allows a sleeping or partial node to fetch the closure relevant to objects it cares about.

5.5 Policy Root

The policy root commits to active scoped rule references and operational policies that are canonically in-force for the scope.

It should cover:

- active ruleset references,
- parameter commitments,
- governance-enacted policy deltas,
- browse policy defaults if canonically set by a DAO.

5.6 Receipt Root

The receipt root commits to operational and economic event evidence.

It should be decomposed as:

```
receipt_root = H(
  execution_receipt_root
  || validation_attestation_root
  || risk_attestation_root
  || settlement_receipt_root
  || retention_attestation_root
)
```

This lane matters for machine-work markets, provider accountability, settlement references, and long-horizon auditability.

5.7 Archive Root

The archive root commits not to all archived bytes themselves, but to archive promises, shard references, object-set commitments, and availability horizons.

It should cover:

- retention commitments,
- shard roots,

- archive object-set commitments,
- pruning markers,
- retrieval policy references.

5.8 Capability Root

The capability root commits to advertised service surfaces.

It should be decomposed as:

```
capability_root = H(
  node_capability_root
  || provider_capability_root
  || fabric_mode_root
  || validation_mode_root
  || archive_mode_root
)
```

This lane is critical if Kapnet becomes a real service economy rather than a closed monolith.

6 Header-Visible Summary Structure

The most useful summary object is a fixed header-visible state summary.

6.1 KapHeaderV1

```
KapHeaderV1 = {
  magic,
  version,
  scope_id,
  epoch,
  prev_header_hash,

  state_root,

  namespace_root,
  object_log_root,
  replay_root,
  dependency_root,
  policy_root,
  receipt_root,
  archive_root,
  capability_root,

  anchor_ref_hash,
  summary_flags,
  reserved,
  signature_or_attestation
}
```

This structure is intended to be:

- compact enough for routine exchange,
- rich enough for operational visibility,
- rigid enough for easy partial synchronization logic.

7 Normative Versus Advisory Fields

The roots listed in the canonical state summary should be consensus-bearing. They must match recomputation from the underlying state structures.

Advisory fields should remain outside that consensus core. Examples include:

- estimated item counts,
- subtree byte size hints,
- provider-specific cache hints,
- expected fetch latency,
- availability scoring.

These are useful but should not affect canonical correctness.

8 Data Structure Recommendations by Lane

Kapnet should not force one state structure onto every lane.

8.1 Sparse Keyed Commitments

These are recommended for:

- namespace registries,
- replay sets,
- policy maps,
- capability maps.

They are good where inclusion and exclusion proofs matter.

8.2 Append-Only Logs

These are recommended for:

- accepted object logs,
- execution receipts,
- validation attestations,
- settlement receipts,
- retention attestations.

They are good where ordering and history matter.

8.3 Root-of-Roots Composition

The top level should bind all lanes together with one deterministic root-of-roots shell. This gives both modularity and one canonical top root.

9 Sleeping Node Runtime: Waking and Fetching State From Providers

This section gives a concrete runtime example.

9.1 Scenario

A node has been asleep for several days. While asleep, several new epochs have passed, the DAO has enacted one policy update, multiple member-addressed submissions have arrived, and the user wants to wake the node and recover only what matters to them without downloading the total corpus.

The node's objective is to:

1. learn the latest state summary,
2. verify state continuity,
3. identify which subroots changed,
4. fetch only relevant proof and object material,
5. reconstruct current local interest state.

9.2 Provider Query Sequence

A balanced wake-up sequence should look like this.

Step 1: Fetch latest summary headers. The sleeping node asks one or more providers for:

- latest `KapHeaderV1`,
- recent summary chain since the node's last known epoch,
- optional proof that the latest summary extends the last known summary.

Step 2: Compare visible subroots. The node compares old and new:

- $\texttt{policy}_r oot$,
- $\texttt{receipt}_r oot$,
- $\texttt{object}_l og_r oot$,
- $\texttt{dependency}_r oot$,
- $\texttt{capability}_r oot$,
- `other lanes of interest.`

If only `capability_root` changed, the node may skip object fetching. If `policy_root` and `receipt_root` changed, it fetches those lanes first.

Step 3: Ask for lane-specific diffs or proofs. The node requests:

- proof of active policy changes,
- proof of new member-addressed submission references,
- receipt diffs for watched work items,
- object log chunk references intersecting watched interests.

Step 4: Resolve dependency closure only for relevant objects. For any newly relevant object, the node requests:

- the object bytes,
- dependency summaries,
- unresolved dependency objects,
- replay-relevant proofs if needed.

Step 5: Rebuild local derived views. The node updates:

- inbox or member-addressed queue,
- DAO feed cursor,
- outstanding work-item receipts,
- local policy cache,
- provider capability cache.

9.3 Pseudo-Runtime Example

```
sleeping_node.wake()

headers = provider.fetch_headers_since(last_known_epoch)

for h in headers:
    verify_header_chain(h)

latest = headers[-1]

changed_lanes = compare_roots(local_summary, latest)

if policy_root in changed_lanes:
    policy_diff = provider.fetch_policy_diff(local_policy_root, latest.policy_root)

if object_log_root in changed_lanes:
    candidate_refs = provider.fetch_object_refs_for_interests(
        watched_scopes,
        watched_members,
        watched_feeds,
```

```
        local_object_cursor
    )

if receipt_root in changed_lanes:
    receipt_refs = provider.fetch_receipt_refs_for_interests(
        watched_work_ids,
        watched_member_ids
    )

objects = provider.fetch_objects(candidate_refs + receipt_refs)
deps = provider.fetch_dependency_closure(objects)

validate_and_apply(objects, deps)

local_summary = latest
```

9.4 Why Header-Visible Subroots Help

Without explicit subroots, the waking node sees only one new opaque state root. Then it must ask providers or indexers what changed. With visible subroots, it knows immediately which state lanes changed and can decide what to fetch.

That is one of the strongest practical arguments for this state design.

10 Responding to Received Member-Addressed Submissions

This section models a common interactive runtime.

10.1 Scenario

A member receives submissions explicitly addressed to them, their delegated agent, or one of their role-bound capability IDs. These submissions may include:

- direct replies,
- work offers,
- governance requests,
- moderation actions,
- settlement notices,
- execution receipts,
- archive retrieval notices.

The node should not need to scan the total object corpus to detect these.

10.2 Recommended Addressing Surface

Member-addressed deliveries should be discoverable through one or more of:

- address index commitments in the object log lane,
- receipt lane references,
- separate inbox or addressee map under a scoped subtree,
- provider-side filtered queries against object tags and addressing fields.

A simple model is to maintain an `inbox_root` inside either the namespace or receipt lane, keyed by recipient principal ID.

10.3 Member Addressing Example

```
inbox_root := sparse_map(
  recipient_principal_id -> ordered_list_commitment_of_object_ids
)
```

Then a node waking for a given member can request only the delta under that principal key.

10.4 Runtime Example

Suppose member M wakes and asks for new addressed submissions.

1. Node fetches latest `receipt_root` and `object_log_root`.
2. Node requests inbox delta for principal M.
3. Provider returns object IDs and inclusion proofs under M's inbox path.
4. Node fetches the corresponding objects and dependency closure.
5. Node validates objects.
6. Node classifies them by local handling policy.
7. Node may emit response objects.

10.5 Pseudo-Runtime Example

```
member_id = local_principal.member_id

inbox_delta = provider.fetch_inbox_delta(
    member_id,
    local_inbox_cursor,
    latest_receipt_root
)

incoming_object_ids = inbox_delta.object_ids
incoming_objects = provider.fetch_objects(incoming_object_ids)
incoming_deps = provider.fetch_dependency_closure(incoming_objects)
```

```
validated = validate_and_classify(incoming_objects, incoming_deps)

for item in validated:
    if item.kind == "governance_request":
        queue_for_member_review(item)
    elif item.kind == "execution_receipt":
        attach_to_work_thread(item)
    elif item.kind == "direct_reply":
        append_to_conversation(item)
    elif item.kind == "settlement_notice":
        verify_settlement_binding(item)
```

10.6 Responding

If the member or agent responds, the response object should usually depend on:

- the addressed submission object ID,
- current relevant policy root if policy-sensitive,
- current replay state if consuming a uniqueness slot,
- any local capability token if agent-bounded.

This keeps the reply machine-readable and conflict-aware.

11 Scrolling DAO Feeds by Set Browse Algorithm Mode

This section models a user or agent scrolling a DAO feed under a declared algorithmic browsing mode.

11.1 Why Browse Mode Matters

A feed is not merely a random list of objects. It is a view over accepted state shaped by one or more ranking or ordering conventions.

Kapnet should distinguish:

- canonical acceptance ordering,
- DAO policy-constrained visibility ordering,
- user-selected browse ordering,
- provider-computed ranking overlays.

The first is closer to consensus. The latter three are browsing behavior and should usually remain outside core correctness.

11.2 Browse Algorithm Modes

A useful minimal set is:

1. `CANONICAL`: accepted-object log order.
2. `RECENT`: reverse-chronological by accepted timestamp or epoch position.
3. `THREAD`: grouped by dependency / reply structure.
4. `RISK_FIRST`: prioritize validated or underwritten items.
5. `MEMBER_RELEVANT`: prioritize items addressing the member or intersecting their watched rooms and roles.
6. `POLICY_WEIGHTED`: apply DAO-scoped weighting rules from the current policy root.
7. `WORK_QUEUE`: prioritize unresolved work objects and receipts.

These should be modeled as *set browse algorithm modes*. The node sets the mode, then queries for the ordered window under that mode.

11.3 Set Browse Model

The node should conceptually browse over a set:

$$S = \{o_1, o_2, \ldots, o_n\}$$

where S is filtered by:

- scope,
- room,
- DAO,
- object type,
- member addressing,
- validation status,
- archive availability.

A browse mode then provides a ranking function B such that:

$$B(S) = \text{ordered window of objects under selected browse mode}$$

11.4 Pseudo-Runtime Example

```
browse_mode = "MEMBER_RELEVANT"

feed_set = provider.fetch_feed_refs(
    dao_id = watched_dao,
    room_ids = watched_rooms,
    object_types = ["submission", "reply", "proposal", "receipt"],
    from_cursor = local_feed_cursor
)

if browse_mode == "CANONICAL":
    ordered = sort_by_canonical_log_position(feed_set)

elif browse_mode == "RECENT":
    ordered = sort_by_recent_acceptance(feed_set)

elif browse_mode == "THREAD":
    ordered = sort_by_dependency_thread(feed_set)

elif browse_mode == "RISK_FIRST":
    ordered = sort_by_validation_and_risk(feed_set)

elif browse_mode == "MEMBER_RELEVANT":
    ordered = sort_by_member_relevance(feed_set, local_member_profile)

elif browse_mode == "POLICY_WEIGHTED":
    ordered = apply_policy_weighting(feed_set, current_policy_snapshot)

elif browse_mode == "WORK_QUEUE":
    ordered = sort_by_unresolved_work_priority(feed_set)
```

11.5 DAO-Set Browse Policy

A DAO may canonically publish default browse-policy references through the policy lane, but individual users and nodes should still be able to locally override browsing behavior unless the application explicitly forbids this.

That means:

- the `policy_root` may commit to default browsing references,
- the user may set a local `browse_mode`,
- providers may expose browse-optimized feed services,
- canonical correctness still depends on accepted objects, not on one universal feed ordering.

12 Integrated Runtime Example

This section ties the previous examples together.

12.1 Scenario

A member's laptop node has been sleeping. On wake:

- it fetches latest state from providers,
- updates current DAO policy,
- fetches member-addressed submissions,
- fetches watched work-item receipts,
- scrolls the DAO feed in `MEMBER_RELEVANT` mode.

12.2 Integrated Pseudocode

```
wake_node()

latest_header = provider.fetch_latest_header(scope_id)
verify_header(latest_header)

changed = compare_roots(local_header, latest_header)

if changed.policy_root:
    policy_state = provider.fetch_policy_state_diff(
        local_header.policy_root,
        latest_header.policy_root
    )
    apply_policy_state(policy_state)

if changed.receipt_root:
    inbox_delta = provider.fetch_inbox_delta(
        member_id,
        local_inbox_cursor,
        latest_header.receipt_root
    )
    receipt_delta = provider.fetch_receipt_refs_for_work_ids(
        watched_work_ids,
        local_receipt_cursor
    )

    incoming_ids = inbox_delta.object_ids + receipt_delta.object_ids
    incoming_objects = provider.fetch_objects(incoming_ids)
    incoming_deps = provider.fetch_dependency_closure(incoming_objects)

    validate_and_apply(incoming_objects, incoming_deps)

if changed.object_log_root:
    feed_refs = provider.fetch_feed_refs(
        dao_id = watched_dao,
        room_ids = watched_rooms,
        object_types = watched_types,
        from_cursor = local_feed_cursor
    )
```

```
    ordered = sort_by_member_relevance(feed_refs, local_member_profile)
    render_feed_window(ordered[0:50])

local_header = latest_header
persist_local_state()
```

13 Why This State Design Supports Provider Optionality

The same state model supports different provider classes cleanly.

13.1 Relay Provider

A relay provider focuses on:

- recent object-log chunks,
- current capability advertisements,
- light feed-serving surfaces.

13.2 Validation Provider

A validation provider focuses on:

- state summary chain,
- dependency proofs,
- replay proofs,
- receipt verification.

13.3 Archive Provider

An archive provider focuses on:

- archive-root material,
- old object chunks,
- historical receipts,
- object-set retention commitments.

13.4 Index and Feed Provider

An indexer or feed provider focuses on:

- browse-mode materialization,
- member relevance indexes,
- thread projections,
- DAO-specific query surfaces.

This is precisely why state lanes should be typed and visible.

14 Minimal Byte-Minded Example

A state summary header for a DAO scope could conceptually look like this.

```
KapHeaderV1 {
  magic = "KAPH"
  version = 1
  scope_id = 0x11...11
  epoch = 0x0000000000000042
  prev_header_hash = 0xaa...aa

  state_root = 0xbb...bb

  namespace_root = 0x01...01
  object_log_root = 0x02...02
  replay_root = 0x03...03
  dependency_root = 0x04...04
  policy_root = 0x05...05
  receipt_root = 0x06...06
  archive_root = 0x07...07
  capability_root = 0x08...08

  anchor_ref_hash = 0x09...09
  summary_flags = 0x00000003
  reserved = 0x00000000
  signature = 0xcc...cc
}
```

If a sleeping node compares this header to its last stored header and sees only `receipt_root` and `object_log_root` changed, then it need not waste time fetching policy or archive state.

15 Implementation Guidance

A practical first implementation should keep the following discipline.

1. Keep the top-level subroot set fixed.
2. Make the subroots header-visible.
3. Use sparse keyed commitments for live maps.
4. Use append-only logs for history lanes.
5. Provide lane-specific proof and diff APIs.
6. Keep browse algorithm mode outside consensus unless a scoped application explicitly commits to one default policy.
7. Let sleeping nodes fetch only changed lanes.
8. Let member-addressed objects be discoverable without total corpus scans.

16 Conclusion

Kapnet should implement state tracking as a typed root-of-roots system with a fixed set of header-visible subroots. This design is superior to a single opaque state root because it provides direct operational visibility, proof modularity, selective synchronization, cleaner provider-market boundaries, and practical support for light or sleeping nodes.

The most important consequence is runtime clarity. A sleeping node can wake, compare subroots, fetch only relevant state lanes, reconstruct member-addressed submissions, and scroll a DAO feed under an explicitly chosen browse algorithm mode without pretending that every byte in the system must always be fetched, replayed, and rendered in one monolithic pass.

That is the correct balance between machine precision and real-world operability.